

Name : Suraj Haridas Jagtap

College Name : Government Polytechnic Achalpur

Domain : AI/ML

Q1. Find a dataset that has at least one continuous target variable (e.g., Price, Salary, Sales, Temperature, House Price, Car Mileage, etc.).

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
```

```
df=pd.read_csv("ford.csv")
```

df

...	model	year	price	transmission	mileage	fuelType	tax	mpg	engineSize	...
0	Fiesta	2017	12000	Automatic	15944	Petrol	150	57.7	1.0	
1	Focus	2018	14000	Manual	9083	Petrol	150	57.7	1.0	
2	Focus	2017	13000	Manual	12456	Petrol	150	57.7	1.0	
3	Fiesta	2019	17500	Manual	10460	Petrol	145	40.3	1.5	
4	Fiesta	2019	16500	Automatic	1482	Petrol	145	48.7	1.0	
...	...	...	...	...	...	...	...	...	...	
17961	B-MAX	2017	8999	Manual	16700	Petrol	150	47.1	1.4	
17962	B-MAX	2014	7499	Manual	40700	Petrol	30	57.7	1.0	
17963	Focus	2015	9999	Manual	7010	Diesel	20	67.3	1.6	
17964	KA	2018	8299	Manual	5007	Petrol	145	57.7	1.2	

Q2. (Dataset Selection & Initial Exploration)

```
df.columns
```

```
Index(['model', 'year', 'price', 'transmission', 'mileage', 'fuelType', 'tax',  
      'mpg', 'engineSize'],  
      dtype='object')
```

```
df.head(10)
```

...	model	year	price	transmission	mileage	fuelType	tax	mpg	engineSize
0	Fiesta	2017	12000	Automatic	15944	Petrol	150	57.7	1.0
1	Focus	2018	14000	Manual	9083	Petrol	150	57.7	1.0
2	Focus	2017	13000	Manual	12456	Petrol	150	57.7	1.0
3	Fiesta	2019	17500	Manual	10460	Petrol	145	40.3	1.5
4	Fiesta	2019	16500	Automatic	1482	Petrol	145	48.7	1.0
5	Fiesta	2015	10500	Manual	35432	Petrol	145	47.9	1.6
6	Puma	2019	22500	Manual	2029	Petrol	145	50.4	1.0
7	Fiesta	2017	9000	Manual	13054	Petrol	145	54.3	1.2
8	Kuga	2019	25500	Automatic	6894	Diesel	145	42.2	2.0
9	Focus	2018	10000	Manual	48141	Petrol	145	61.4	1.0

I chose this used cars dataset because it has simple and practical features like mileage, year, fuel type, and engine size that directly affect car prices. It includes both categorical features (like transmission and fuelType) and numerical features (like mileage, mpg, engineSize), which makes it good for regression and feature analysis. The target variable is price, which is continuous and important for predicting car values. This dataset helps us see how different car details impact resale value, making it useful for learning data analysis and machine learning in an easy, real-world way.

Q3. (Missing & Duplicate Values Analysis)

```
▶ missing = df.isnull().sum()
missing_percent = (missing / len(df)) * 100
print(missing_percent)
df = df.dropna()
df = df.drop_duplicates()
print(df.head(10))
```

```
... model          0.0
    year          0.0
    price         0.0
    transmission  0.0
    mileage       0.0
    fuelType      0.0
    tax           0.0
    mpg           0.0
    engineSize    0.0
    dtype: float64
   model  year  price  transmission  mileage  fuelType  tax  mpg  engineSize
0  Fiesta  2017  12000    Automatic    15944    Petrol   150  57.7         1.0
1  Focus   2018  14000     Manual     9083    Petrol   150  57.7         1.0
2  Focus   2017  13000     Manual    12456    Petrol   150  57.7         1.0
3  Fiesta  2019  17500     Manual    10460    Petrol   145  40.3         1.5
4  Fiesta  2019  16500    Automatic    1482    Petrol   145  48.7         1.0
5  Fiesta  2015  10500     Manual   35432    Petrol   145  47.9         1.6
6  Puma    2019  22500     Manual    2029    Petrol   145  50.4         1.0
7  Fiesta  2017   9000     Manual   13054    Petrol   145  54.3         1.2
8  Kuga    2019  25500    Automatic    6894    Diesel   145  42.2         2.0
9  Focus   2018  10000     Manual   48141    Petrol   145  61.4         1.0
```

I checked each column for missing values and calculated their percentage. Some columns had missing values, so I dropped those rows to keep the dataset clean. I also removed duplicate rows to avoid repetition. After cleaning, the dataset is ready for analysis and modeling. The target variable **price** is now more reliable because missing or duplicate data could affect predictions.

Q4. (Statistical Summary)

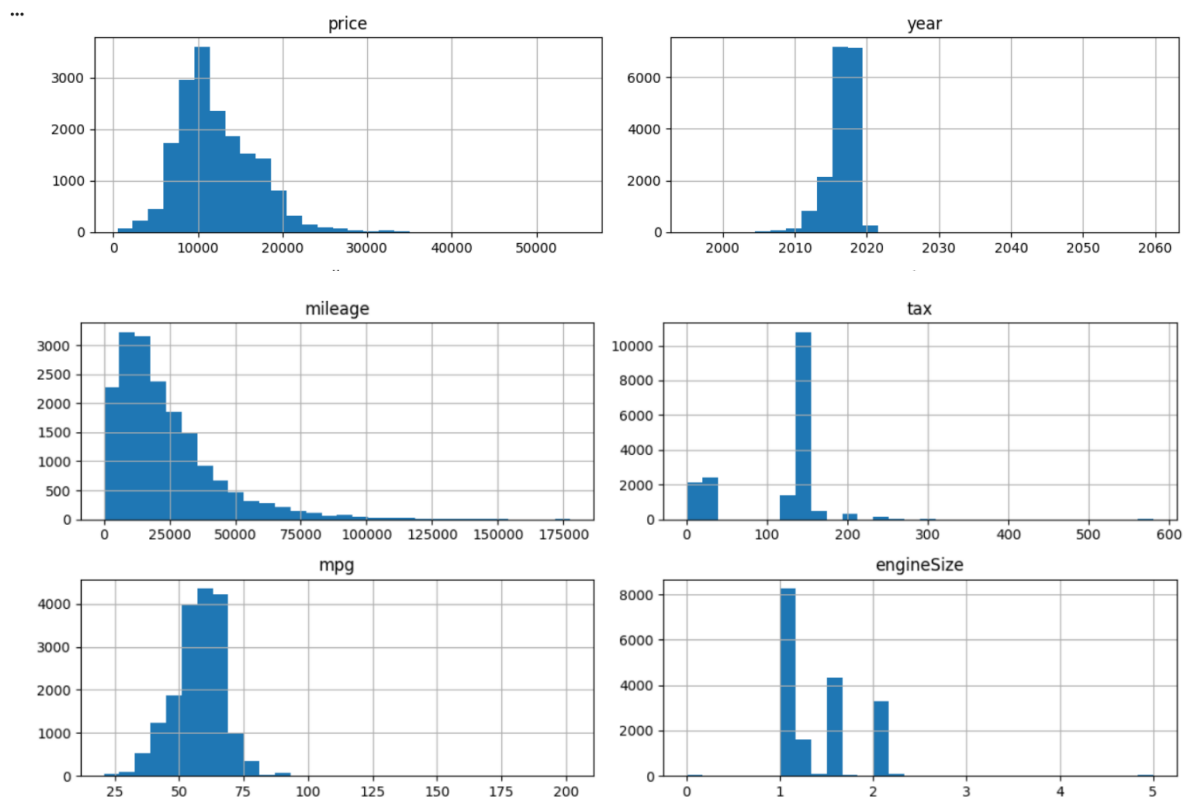
```
▶ df.describe()  
df["price"].min()  
df["price"].max()  
df["price"].mean()  
df["price"].median()
```

```
... 11288.0
```

Car prices go from very low to very high, showing a big range in the dataset. The mean price is higher than the median, which means a few very expensive cars are raising the average. Mileage also varies a lot, so the dataset has both new and old cars. Engine size and mpg are diverse, covering small fuel-efficient cars and large powerful ones.

Q5. (Histogram Analysis of Numeric Features)

```
numeric_cols = ["price", "year", "mileage", "tax", "mpg", "engineSize"]
df[numeric_cols].hist(bins=30, figsize=(12,8))
plt.tight_layout()
plt.show()
```



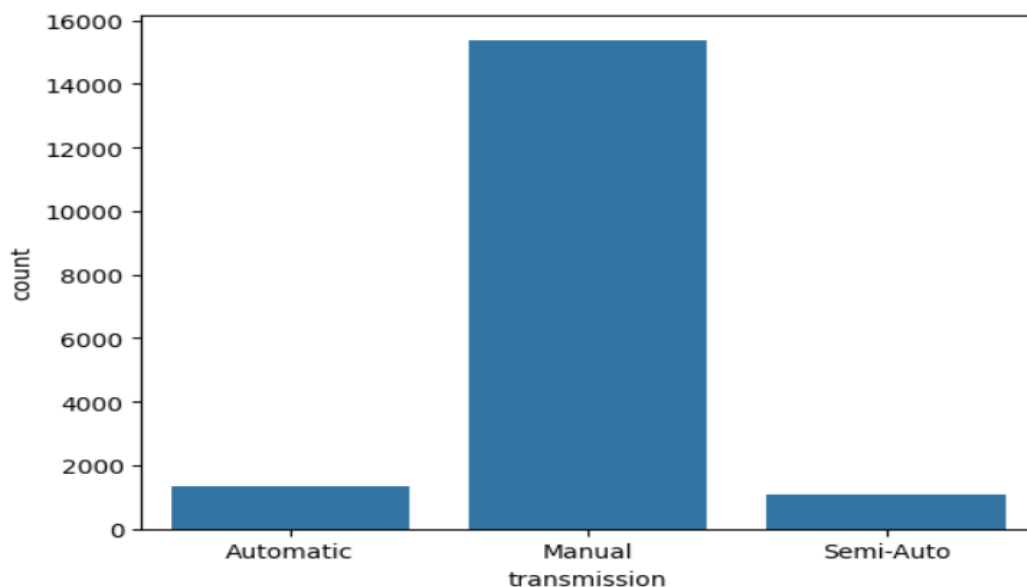
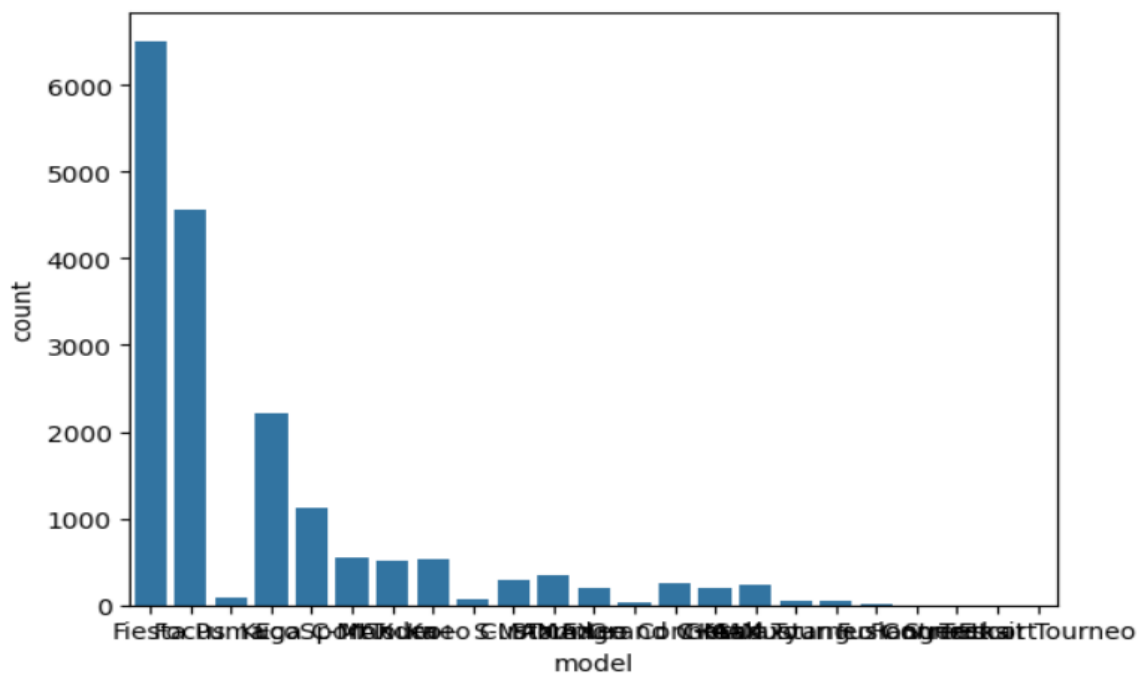
- **Price** → Right-skewed; most cars are low-priced, few are very expensive.
- **Year** → Concentrated around recent years; dataset mostly has newer cars.
- **Mileage** → Many cars have low mileage, but some very high values exist.
- **Tax** → Clustered around fixed amounts, showing standard tax brackets.
- **MPG** → Most cars have average fuel efficiency, fewer extremes.
- **Engine Size** → Majority are small to medium engines, fewer large ones.

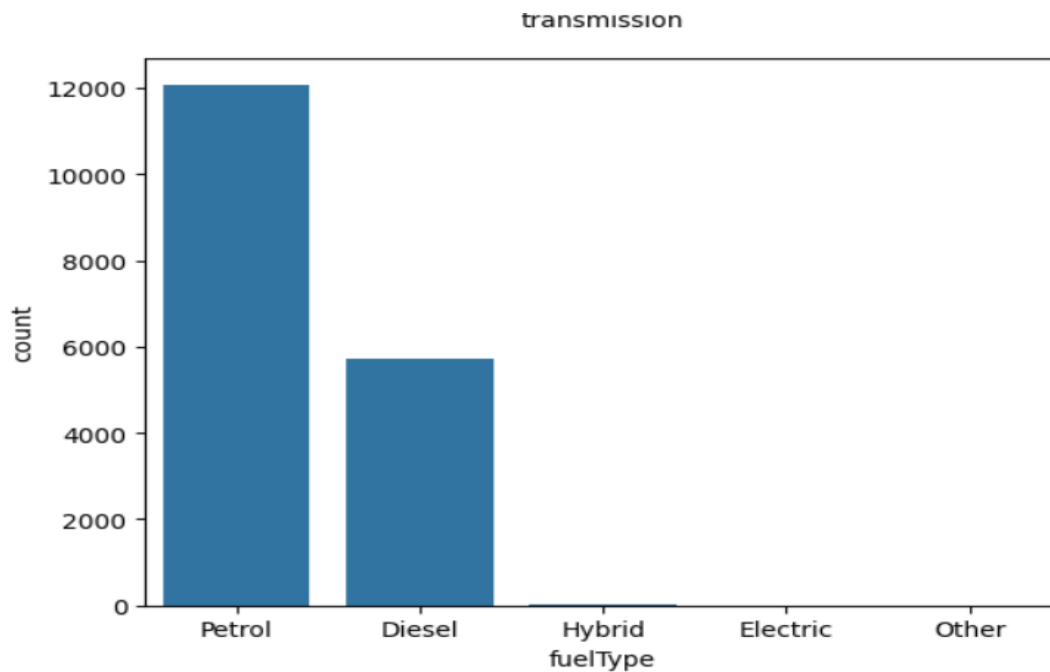
Q6. (Count Plots of Categorical Features)

```

categorical_cols = ["model", "transmission", "fuelType"]
sns.countplot(x="model", data=df)
plt.show()
sns.countplot(x="transmission", data=df)
plt.show()
sns.countplot(x="fuelType", data=df)
plt.show()

```

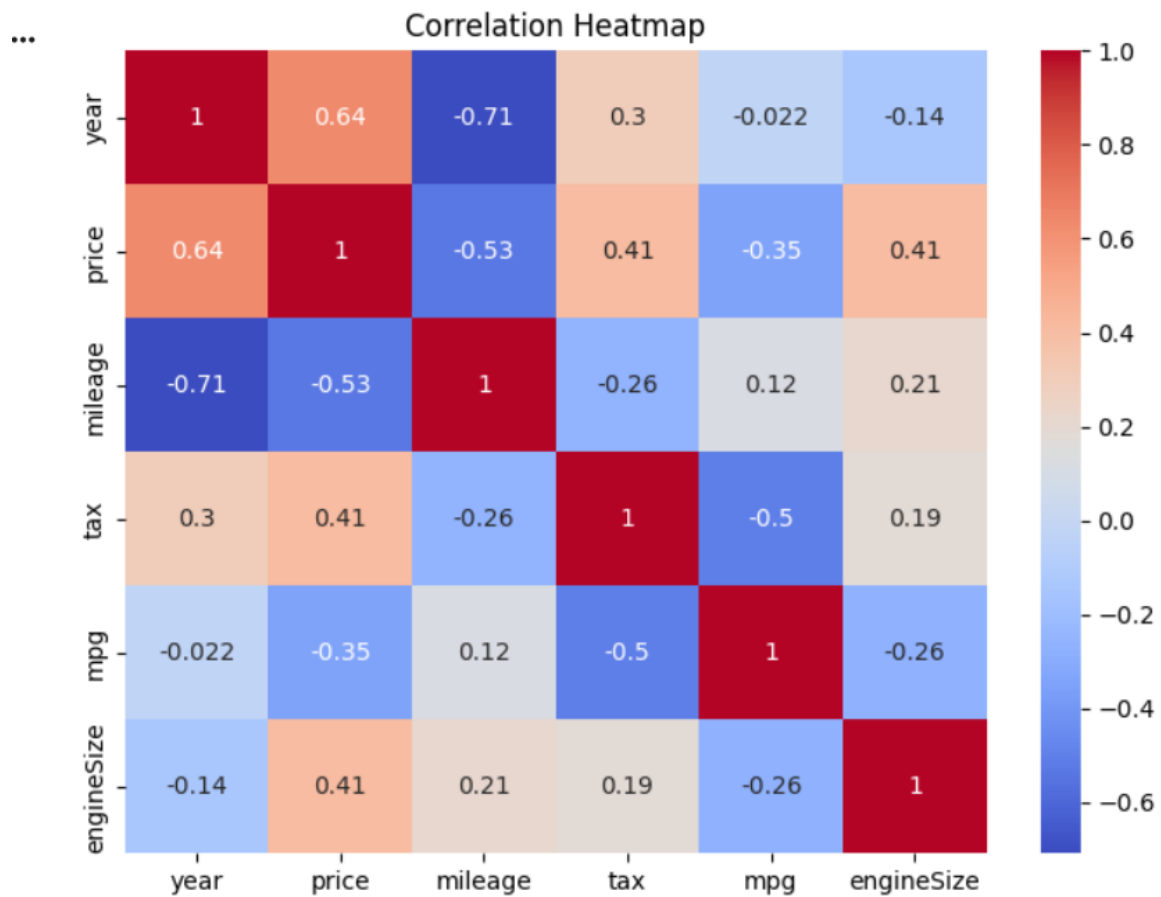




- **Model** → Some models appear more often, meaning they are more popular in the dataset.
- **Transmission** → Automatic or manual preference shows customer demand trends.
- **Fuel Type** → Petrol cars dominate, while diesel and hybrid are fewer, showing market shift toward efficiency.

Q7. (Correlation Heatmap)

```
numeric_df = df.select_dtypes(include=["int64", "float64"])
plt.figure(figsize=(8,6))
sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



Q8. (Feature Identification) Clearly define:

- **Independent Features (X)** → model, year, transmission, mileage, fuelType, tax, mpg, engineSize
- **Dependent Feature (y)** → price

Q9. (Encoding Categorical Variables) Convert all categorical columns to numeric form using suitable encoding techniques (One-Hot Encoding).

```
df_encoded = pd.get_dummies(df, columns=["transmission", "fuelType"])
```

```
print(df_encoded.filter(like="transmission").head())
print(df_encoded.filter(like="fuelType").head())
```

```
... transmission_Automatic transmission_Manual transmission_Semi-Auto
0 True False False
1 False True False
2 False True False
3 False True False
4 True False False
fuelType_Diesel fuelType_Electric fuelType_Hybrid fuelType_Other \
0 False False False False
1 False False False False
2 False False False False
3 False False False False
4 False False False False
fuelType_Petrol
0 True
1 True
2 True
3 True
4 True
```

- I used **One-Hot Encoding** because categorical features like transmission and fuelType are non-numeric.
- One-Hot Encoding creates new binary columns (0/1) for each category, which avoids giving false numerical meaning to categories.
- Example: transmission becomes transmission_Manual, transmission_Automatic, etc.
- Example: fuelType becomes fuelType_Petrol, fuelType_Diesel, fuelType_Hybrid, etc.
- This makes the dataset suitable for machine learning models, which require numeric inputs.

Q10. (Feature Scaling) Apply Standard Scaling on the independent features using StandardScaler.

```
▶ x = df[["year", "mileage", "tax", "mpg", "engineSize"]]  
  
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)  
  
print(X_scaled[:5])
```

```
... [[ 0.06705901 -0.38299351  0.59137991 -0.02059691 -0.81056084]  
      [ 0.55439275 -0.73631699  0.59137991 -0.02059691 -0.81056084]  
      [ 0.06705901 -0.56261635  0.59137991 -0.02059691 -0.81056084]  
      [ 1.0417265  -0.66540511  0.51077747 -1.73785836  0.34532516]  
      [ 1.0417265  -1.12774853  0.51077747 -0.90883559 -0.81056084]]
```

❖ <https://github.com/surajjagtap08062007-sj/surajpython.git>